

Small Virtual Network with DNS and Web Services using GNS3

Jose Quintero

JAQ0037

CSCE 3530

Dr. Ali Zarafshani

7/11/2026

1. Project Overview

The purpose of this project was to design and build a small virtual network using GNS3. The network demonstrates common networking protocols such as DHCP, DNS, HTTP, ICMP, TCP, UDP, NAT, and IP addressing. A Windows virtual machine was configured as the client while Linux servers provided DNS and web services. Wireshark was used to verify network traffic and protocol communication.

2. Objectives

- Design a functional virtual network
- Configure IP addressing
- Configure a DNS server
- Configure a web server
- Test connectivity
- Capture network traffic with Wireshark
- Demonstrate networking protocols

3. Network Design Explanation

The network used in this project is created through GNS3 and is designed to represent the network of a small business firm. The structure of the network includes a NAT cloud, Ethernet switch, Windows virtual machine, and Ubuntu Server hosting the DNS and Nginx web server services. The NAT cloud serves as a provider of Internet connection for the virtual network, whereas the Ethernet switch connects all the network devices together.

In this arrangement, the Windows virtual machine is the client machine. It is assigned an IP address and interacts with the Ubuntu server to conduct DNS lookups and gain access to the webpage hosted on the server. The Ubuntu server is capable of providing two significant network services. Firstly, the Ubuntu server serves as a DNS server, converting domain names to IP addresses to enable clients to find their resources on the network. Secondly, it operates a web server via Nginx, which enables the Windows client to connect to a web page.

All the devices have been interconnected using the virtual switch and all devices are allocated IP addresses that belong to the same subnet, which makes it possible for direct communication among the devices. In case the Windows client needs a particular website, the first thing it does is that it performs a DNS lookup to the Ubuntu server in order to get the IP address of that website domain name. The client then creates a TCP connection to the web server and gets the webpage using the HTTP protocol. The ICMP protocol has been used in our project to check connectivity between devices using the ping command.

This network design demonstrates how multiple networking protocols and services work together in a real-world environment. By combining DNS, HTTP, TCP/IP, ICMP, and NAT into a single virtual network, the project provides a practical understanding of how clients locate servers, establish connections, and exchange data across a network.

4. Description of Networking Protocols Used

IP Addressing

IP addresses were used to give each device on the network its own unique address. This allowed the Windows client and Ubuntu server to find each other and communicate. Without IP addresses, the devices wouldn't know where to send or receive data.

ARP (Address Resolution Protocol)

ARP helped devices find each other's MAC addresses before sending data across the local network. When the Windows client wanted to contact the Ubuntu server, ARP matched the server's IP address to its physical MAC address so the data could be delivered correctly.

ICMP (Internet Control Message Protocol)

ICMP was mainly used to test if the devices could communicate by using the ping command. If the ping was successful, it confirmed that the network was working properly. This was one of the main tools used to troubleshoot connection issues during the project.

TCP (Transmission Control Protocol)

TCP handled the communication between the Windows client and the web server. It made sure all the data arrived in the correct order and that nothing was lost during transmission. This made it a good choice for loading webpages.

UDP (User Datagram Protocol)

UDP was used by the DNS server to answer name lookup requests. Since DNS messages are usually small and need quick responses, UDP works well because it doesn't spend extra time creating a connection before sending data.

DNS (Domain Name System)

The Ubuntu server was set up as the DNS server for the network. Its job was to translate domain names into IP addresses. Instead of typing an IP address into the browser, the Windows client could use a hostname, and the DNS server would tell it where to connect.

HTTP (Hypertext Transfer Protocol)

HTTP was used when the Windows client accessed the webpage hosted on the Ubuntu server. After the DNS server provided the correct IP address, HTTP handled sending the webpage from the server to the client so it could be displayed in the browser.

NAT (Network Address Translation)

NAT allowed the virtual network to connect to the Internet using the GNS3 NAT Cloud. It translated the private IP addresses used inside the virtual network into a public address whenever Internet access was needed. This allowed the virtual machines to reach the Internet while keeping the internal network separate.

5. Device configurations

Windows 10 Client

- Obtained IPv4 address automatically through DHCP
- DNS server configured as 192.168.42.180
- Default gateway assigned by the GNS3 NAT Cloud
- Used for DNS lookups, web browsing, and network testing

Ubuntu Server

- Ubuntu Server installed in GNS3
- Static IP address: 192.168.42.180/24
- BIND9 configured as the DNS server
- Nginx configured as the HTTP web server

GNS3 NAT Cloud

- Provides Internet access
- Supplies DHCP addresses to the Windows client
- Performs Network Address Translation (NAT)

Device	Operating System	IP Address	Purpose
Windows 10 VM	Windows 10	DHCP Assigned	Client Workstation
Ubuntu Server	Ubuntu Server	192.168.42.180	DNS & Web Server
Ethernet Switch	GNS3 Ethernet Switch	N/A	Layer 2 Switching
NAT Cloud	GNS3 NAT	192.168.42.1	DHCP & Internet Access

6. Testing and Verification

Screenshot 1 -Final GNS3 Topology

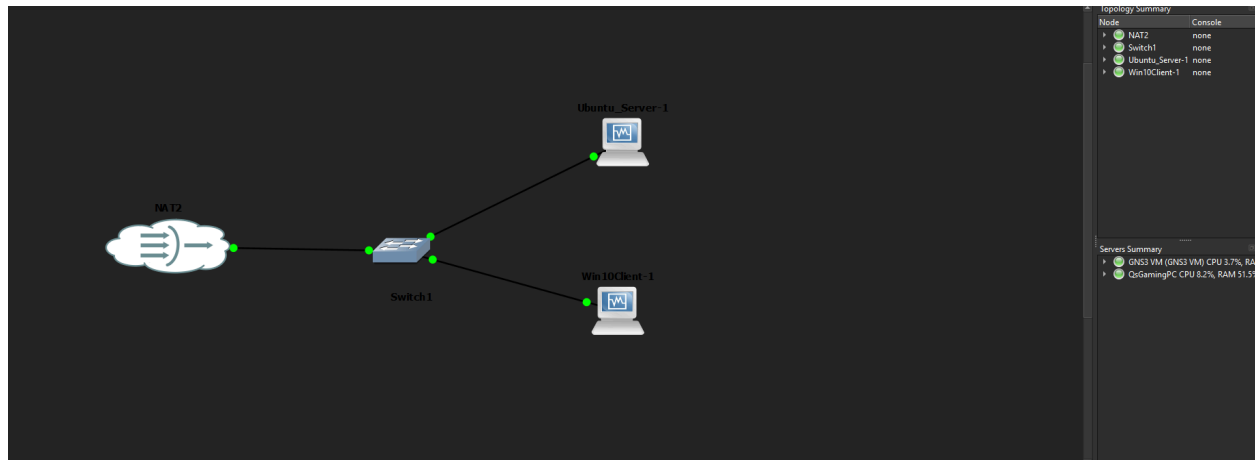


Figure 1. Network Topology. GNS3 network topology showing the NAT device, Ethernet switch, Ubuntu DNS server, and Windows 10 client.

Screenshot 2 — Windows IP Configuration

```
Command Prompt
C:\Users\adminjose>ipconfig /all

Windows IP Configuration

    Host Name . . . . . : Win10Client
    Primary Dns Suffix . . . . . :
    Node Type . . . . . : Hybrid
    IP Routing Enabled. . . . . : No
    WINS Proxy Enabled. . . . . : No

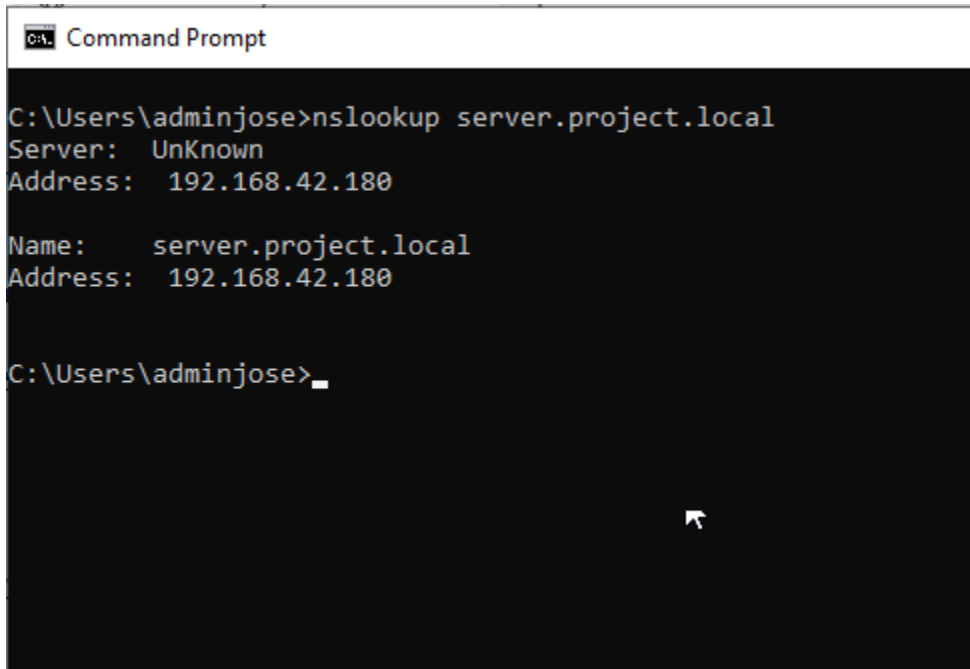
Ethernet adapter Ethernet:

    Connection-specific DNS Suffix . :
    Description . . . . . : Intel(R) PRO/1000 MT Desktop Adapter
    Physical Address. . . . . : 08-00-27-07-EB-A0
    DHCP Enabled. . . . . : Yes
    Autoconfiguration Enabled . . . . : Yes
    Link-local IPv6 Address . . . . . : fe80::d525:e29:8569:b974%4(Preferred)
    IPv4 Address. . . . . : 192.168.42.242(Preferred)
    Subnet Mask . . . . . : 255.255.255.0
    Lease Obtained. . . . . : Sunday, July 12, 2026 8:44:42 AM
    Lease Expires . . . . . : Sunday, July 12, 2026 10:31:57 AM
    Default Gateway . . . . . : 192.168.42.1
    DHCP Server . . . . . : 192.168.42.1
    DHCPv6 IAID . . . . . : 101187623
    DHCPv6 Client DUID. . . . . : 00-01-00-01-31-E4-E7-E4-08-00-27-07-EB-A0
    DNS Servers . . . . . : 192.168.42.180
    NetBIOS over Tcpi . . . . . : Enabled

C:\Users\adminjose>
```

Figure 2. Windows DHCP Configuration. The Windows 10 client automatically received its IP address, subnet mask, default gateway, and DNS server through DHCP.

Screenshot 3 – DNS Resolution

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt". The command prompt shows the user 'adminjose' at the 'C:\Users\' directory. The command 'nslookup server.project.local' has been entered. The output shows 'Server: UnKnown' and 'Address: 192.168.42.180'. Below this, it shows 'Name: server.project.local' and 'Address: 192.168.42.180'. The prompt is now waiting for the next command.

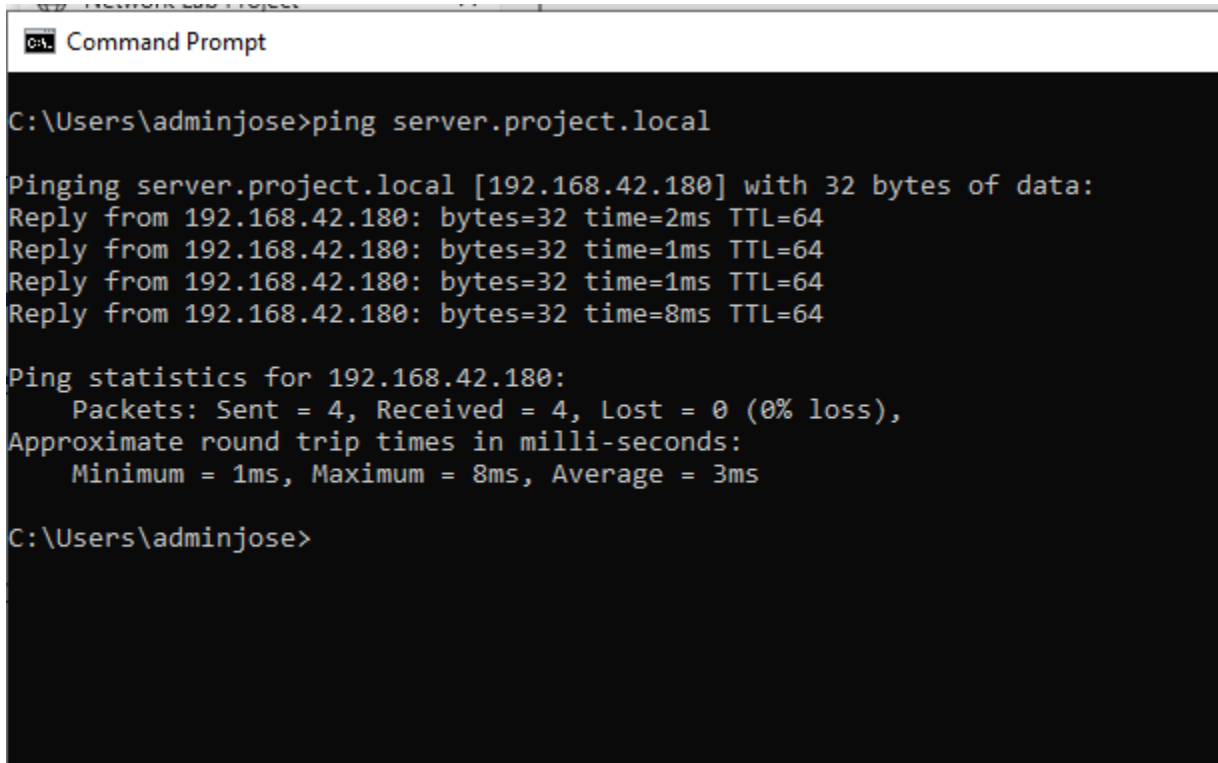
```
C:\Users\adminjose>nslookup server.project.local
Server:  UnKnown
Address: 192.168.42.180

Name:    server.project.local
Address: 192.168.42.180

C:\Users\adminjose>
```

Figure 3. DNS name resolution using the Ubuntu DNS server. The Windows 10 client successfully queried the Ubuntu DNS server (192.168.42.180) and resolved the hostname server.project.local to its corresponding IP address, confirming that DNS is functioning correctly.

Screenshot 4 – Ping by Hostname

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt". The command prompt shows the user typing "ping server.project.local" at the C:\Users\adminjose> prompt. The output shows four successful replies from 192.168.42.180 with varying times (2ms, 1ms, 1ms, 8ms) and a TTL of 64. Ping statistics are also displayed, showing 4 packets sent, 4 received, and 0% loss, with an average round trip time of 3ms.

```
C:\Users\adminjose>ping server.project.local

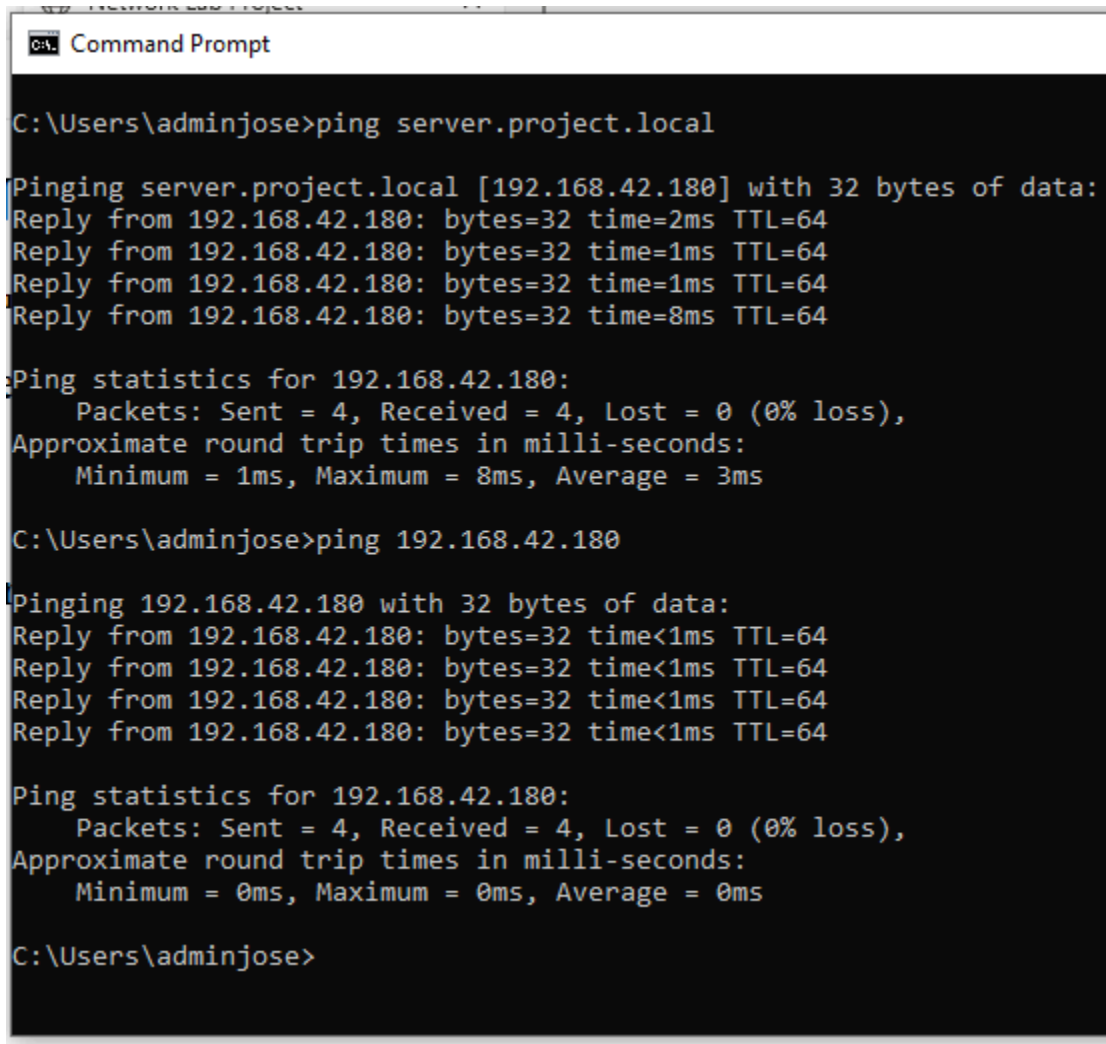
Pinging server.project.local [192.168.42.180] with 32 bytes of data:
Reply from 192.168.42.180: bytes=32 time=2ms TTL=64
Reply from 192.168.42.180: bytes=32 time=1ms TTL=64
Reply from 192.168.42.180: bytes=32 time=1ms TTL=64
Reply from 192.168.42.180: bytes=32 time=8ms TTL=64

Ping statistics for 192.168.42.180:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 8ms, Average = 3ms

C:\Users\adminjose>
```

Figure 4. Ping test using the DNS hostname. The Windows 10 client successfully resolved server.project.local through the DNS server and communicated with the Ubuntu server using ICMP, confirming both DNS name resolution and network connectivity.

Screenshot 5 – Ping by IP



```
C:\Users\adminjose>ping server.project.local

Pinging server.project.local [192.168.42.180] with 32 bytes of data:
Reply from 192.168.42.180: bytes=32 time=2ms TTL=64
Reply from 192.168.42.180: bytes=32 time=1ms TTL=64
Reply from 192.168.42.180: bytes=32 time=1ms TTL=64
Reply from 192.168.42.180: bytes=32 time=8ms TTL=64

Ping statistics for 192.168.42.180:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 8ms, Average = 3ms

C:\Users\adminjose>ping 192.168.42.180

Pinging 192.168.42.180 with 32 bytes of data:
Reply from 192.168.42.180: bytes=32 time<1ms TTL=64
Reply from 192.168.42.180: bytes=32 time<1ms TTL=64
Reply from 192.168.42.180: bytes=32 time<1ms TTL=64
Reply from 192.168.42.180: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.42.180:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\adminjose>
```

Figure 5. ICMP connectivity test using the server's IP address. The Windows 10 client successfully communicated with the Ubuntu server using its IP address. Four successful ICMP echo replies with 0% packet loss confirm that network connectivity between the client and server is functioning correctly.

Screenshot 6 – Ubuntu Network Configuration

```
adminjose@dns-server:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:a3:a8:42 brd ff:ff:ff:ff:ff:ff
    altname enx080027a3a842
    inet 192.168.42.180/24 metric 1024 brd 192.168.42.255 scope global dynamic enp0s3
        valid_lft 2933sec preferred_lft 2933sec
    inet6 fe80::a00:27ff:fea3:a842/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
adminjose@dns-server:~$
```

Figure 6. Ubuntu Server network configuration. The ip addr command confirms that the Ubuntu server's network interface (enp0s3) is active and configured with the IPv4 address 192.168.42.180/24, allowing it to communicate with the Windows client and provide network services.

Screenshot 7 – DNS Zone Configuration

```
adminjose@dns-server:~$ cat /etc/bind/db.project.local
$TTL 604800
@      IN      SOA      ns1.project.local. admin.project.local. (
                                2
                                604800
                                86400
                                2419200
                                604800
)

@      IN      NS       ns1.project.local.
@      IN      A        192.168.42.180
ns1    IN      A        192.168.42.180
server IN      A        192.168.42.180
adminjose@dns-server:~$
```

Figure 7. DNS zone file configuration. The BIND9 DNS zone file (db.project.local) defines the project.local domain, including the Start of Authority (SOA) record, the Name Server (NS) record, and A records that map ns1.project.local and server.project.local to the Ubuntu server's IP address (192.168.42.180).

Screenshot 8 – Verify the DNS Service Is Running

```
adminjose@dns-server:~$ sudo systemctl status bind9
[sudo: authenticate] Password:
• named.service - BIND Domain Name Server
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-07-12 14:13:59 UTC; 42min ago
   Invocation: 38c08d41939b4b3896a4c940f0bc38ec
     Docs: man:named(8)
    Main PID: 3116 (named)
     Status: "running"
      Tasks: 8 (limit: 1730)
     Memory: 10.2M (peak: 10.4M)
        CPU: 170ms
    CGroup: /system.slice/named.service
            └─3116 /usr/sbin/named -f -u bind

Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'data.trafficmanager.net/NS/IN': 2a01:111:4000:f00::f0#53
Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'onedscolprdeus10.eastus.cloudapp.azure.com/A/IN': 2620:1ec:bda:700::c9#53
Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'ns1.a-msedge.net/AAAA/IN': 2001:500:9f::42#53
Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'ns2.a-msedge.net/AAAA/IN': 2001:500:9f::42#53
Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'ns1.a-msedge.net/AAAA/IN': 2001:7fd::1#53
Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'ns2.a-msedge.net/AAAA/IN': 2001:7fd::1#53
Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'ns1.a-msedge.net/AAAA/IN': 2001:503:ba3e::2:30#53
Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'ns2.a-msedge.net/AAAA/IN': 2001:503:ba3e::2:30#53
Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'ns1.a-msedge.net/AAAA/IN': 2001:500:2::c#53
Jul 12 14:55:33 dns-server named[3116]: network unreachable resolving 'ns2.a-msedge.net/AAAA/IN': 2001:500:2::c#53
adminjose@dns-server:~$
```

Figure 8. BIND9 DNS service status. The `systemctl status bind9` command confirms that the BIND9 DNS service is active, running, and enabled to start automatically. This verifies that the Ubuntu server is successfully providing DNS services to the network.

Screenshot 9 – DHCP Configuration

```
Ethernet adapter Ethernet:

    Connection-specific DNS Suffix . . . : 
    Description . . . . . : Intel(R) PRO/1000 MT Desktop Adapter
    Physical Address. . . . . : 08-00-27-07-EB-A0
    DHCP Enabled. . . . . : Yes
    Autoconfiguration Enabled . . . . : Yes
    Link-local IPv6 Address . . . . . : fe80::d525:e29:8569:b974%4(Preferred)
    IPv4 Address. . . . . : 192.168.42.242(Preferred)
    Subnet Mask . . . . . : 255.255.255.0
    Lease Obtained. . . . . : Sunday, July 12, 2026 8:44:42 AM
    Lease Expires . . . . . : Sunday, July 12, 2026 10:31:58 AM
    Default Gateway . . . . . : 192.168.42.1
    DHCP Server . . . . . : 192.168.42.1
    DHCPv6 IAID . . . . . : 101187623
    DHCPv6 Client DUID. . . . . : 00-01-00-01-31-E4-E7-E4-08-00-27-07-EB-A0
    DNS Servers . . . . . : 192.168.42.180
    NetBIOS over Tcpip. . . . . : Enabled

C:\Users\adminjose>
```

Figure 9. Windows 10 network configuration obtained through DHCP. The Windows 10 client successfully received its network configuration from the GNS3 NAT DHCP server, including an IP address, subnet mask, and default gateway. The client is also configured to use the Ubuntu server (192.168.42.180) as its DNS server for hostname resolution.

Screenshot 10 - Show Internet Access through NAT

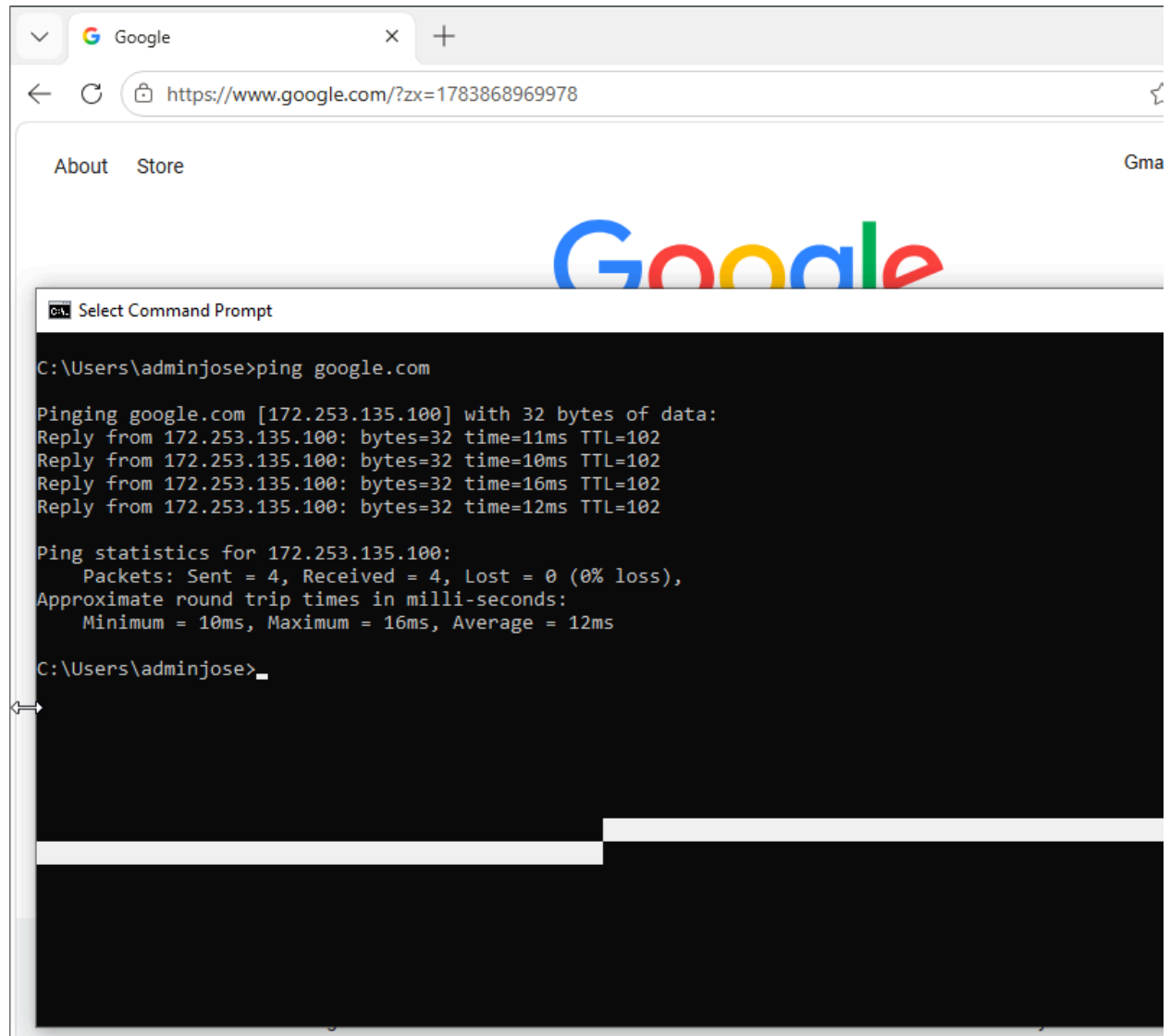


Figure 10. Internet Connectivity Verification. The Windows 10 client successfully accessed the Google website and resolved google.com while receiving four successful ICMP replies. This confirms that DNS resolution, NAT, and Internet connectivity are functioning correctly within the virtual network.

Screenshot 11 - DHCP traffic

Capturing from Standard input [Switch1 Ethernet1 to Win10Client-1 Ethernet0]

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

bootp

No.	Time	Source	Destination	Protocol	Length	Info
250	127.621230	192.168.42.242	192.168.42.1	DHCP	342	DHCP Release - Transaction ID 0x5403d637
314	139.137614	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0xb27b7dde
325	142.160704	192.168.42.1	192.168.42.242	DHCP	342	DHCP Offer - Transaction ID 0xb27b7dde
326	142.165105	0.0.0.0	255.255.255.255	DHCP	362	DHCP Request - Transaction ID 0xb27b7dde
327	142.168161	192.168.42.1	192.168.42.242	DHCP	350	DHCP ACK - Transaction ID 0xb27b7dde

Frame 250: Packet, 342 bytes on wire (2736 bits), 342 bytes captured (2736 bits) on interface 0

Ethernet II, Src: PCSSystemtec 07:eb:a0 (08:00:27:07:eb:a0), Dst: 52:54:00:b8:64:80

Internet Protocol Version 4, Src: 192.168.42.242, Dst: 192.168.42.1

User Datagram Protocol, Src Port: 68, Dst Port: 67

Dynamic Host Configuration Protocol (Release)

0000 52 54 00 b8 64 80 08 00 27 07 eb a0 08 00 45 00 RT...d...E...
0010 01 48 71 57 00 00 80 11 f2 09 c0 a8 2a f2 c0 a8 HqW...*...
0020 2a 01 00 44 00 43 01 34 2d 4d 01 01 06 00 54 03 *..D C 4 -M...T...
0030 d6 37 00 00 00 00 c0 a8 2a f2 00 00 00 00 00 00 -7...*...
0040 00 00 00 00 00 00 08 00 27 07 eb a0 00 00 00 00
0050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00a0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00b0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00d0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00f0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0100 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0110 00 00 00 00 00 00 63 82 53 63 35 01 07 36 04 c0c Sc5 6...
0120 a8 2a 01 3d 07 01 08 00 27 07 eb a0 ff 00 00 00*=...*...
0130 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0140 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Figure 11. DHCP lease negotiation between the Windows 10 client and the Ubuntu DHCP server. The client broadcasts a Discover message, receives an Offer, sends a Request, and receives an Acknowledgment (ACK), completing the DORA process.

Screenshot 12 - Capturing DNS Traffic

Capturing from Standard input [Switch1 Ethernet1 to Win10Client-1 Ethernet0]

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

dns

No.	Time	Source	Destination	Protocol	Length	Info
180	118.348395	192.168.42.242	192.168.42.180	DNS	86	Standard query 0x1e72 A checkappexec.microsoft.com
183	118.446772	192.168.42.180	192.168.42.242	DNS	207	Standard query response 0x1e72 A checkappexec.microsoft.com
357	142.739100	192.168.42.242	192.168.42.180	DNS	76	Standard query 0x904a A dns.msftncsi.com
358	142.817841	192.168.42.180	192.168.42.242	DNS	92	Standard query response 0x904a A dns.msftncsi.com A 131.
366	143.276191	192.168.42.242	192.168.42.180	DNS	86	Standard query 0x7c50 A licensing.mp.microsoft.com
368	143.364010	192.168.42.242	192.168.42.180	DNS	82	Standard query 0x15be A client.wns.windows.com
369	143.393534	192.168.42.180	192.168.42.242	DNS	141	Standard query response 0x15be A client.wns.windows.com
384	143.567135	192.168.42.180	192.168.42.242	DNS	284	Standard query response 0x7c50 A licensing.mp.microsoft.com
542	201.205075	192.168.42.242	192.168.42.180	DNS	92	Standard query 0x08f9 A mobile.events.data.microsoft.com
543	201.285460	192.168.42.180	192.168.42.242	DNS	215	Standard query response 0x08f9 A mobile.events.data.microsoft.com
676	291.111894	192.168.42.242	192.168.42.180	DNS	87	Standard query 0x0001 PTR 180.42.168.192.in-addr.arpa
677	291.112757	192.168.42.180	192.168.42.242	DNS	142	Standard query response 0x0001 No such name PTR 180.42.168.192.in-addr.arpa
678	291.113565	192.168.42.242	192.168.42.180	DNS	68	Standard query 0x0002 A goog.com
679	291.275538	192.168.42.180	192.168.42.242	DNS	100	Standard query response 0x0002 A goog.com A 34.210.62.222
680	291.281666	192.168.42.242	192.168.42.180	DNS	68	Standard query 0x0003 AAAA goog.com
681	291.362967	192.168.42.180	192.168.42.242	DNS	150	Standard query response 0x0003 AAAA goog.com SOA ns-1051

Frame 183: Packet, 207 bytes on wire (1656 bits), 207 bytes captured on interface 0

Ethernet II, Src: PCSSystemtec_a3:a8:42 (08:00:27:a3:a8:42), Dst: 08:00:27:a3:a8:42

Internet Protocol Version 4, Src: 192.168.42.180, Dst: 192.168.42.242

User Datagram Protocol, Src Port: 53, Dst Port: 53755

Domain Name System (response)

0000 08 00 27 07 eb a0 08 00 27 a3 a8 42 08 00 45 00

0010 00 c1 a3 c7 00 00 40 11 ff 6d c0 a8 2a b4 c0 a8@..

0020 2a f2 00 35 d1 fb 00 ad b2 d1 1e 72 81 80 00 01 *.5....

0030 00 03 00 00 00 00 0c 63 68 65 63 6b 61 70 70 65c..

0040 78 65 63 09 6d 69 63 72 6f 73 6f 66 74 03 63 6f xec:micr..

0050 6d 00 00 01 00 01 c0 0c 00 05 00 01 00 00 0e 10 m.....

0060 00 28 13 70 72 6f 64 2d 61 74 6d 2d 77 64 73 2d (.prod-..

0070 61 70 70 72 65 70 0e 74 72 61 66 66 69 63 6d 61 apprep:t..

0080 6e 61 67 65 72 03 6e 65 74 00 c0 38 00 05 00 01 nager:ne..

0090 00 00 01 2c 00 29 0f 70 72 6f 64 2d 61 67 69 63 ..,)p....

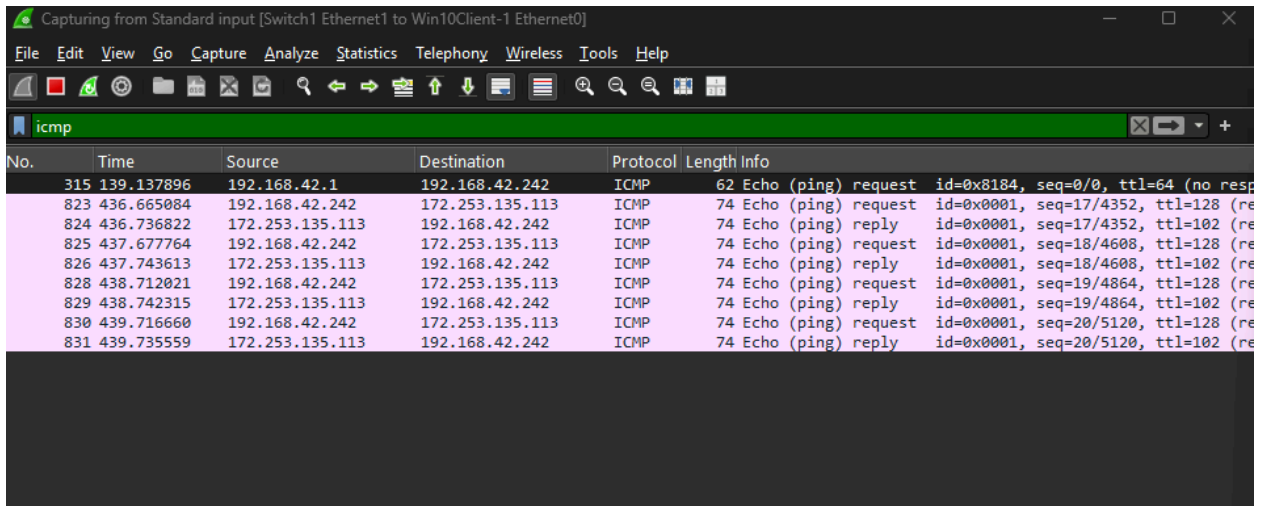
00a0 2d 65 75 32 2d 31 07 65 61 73 74 75 73 32 08 63 -eu2-1.e..

00b0 6c 6f 75 64 61 70 70 05 61 7a 75 72 65 c0 23 c0 loudapp..

00c0 6c 00 01 00 01 00 00 00 0a 00 04 04 99 1d 34 l.....

Figure 12. DNS query and response captured in Wireshark. The Windows 10 client sends DNS requests to the Ubuntu DNS server, which returns the corresponding IP address records.

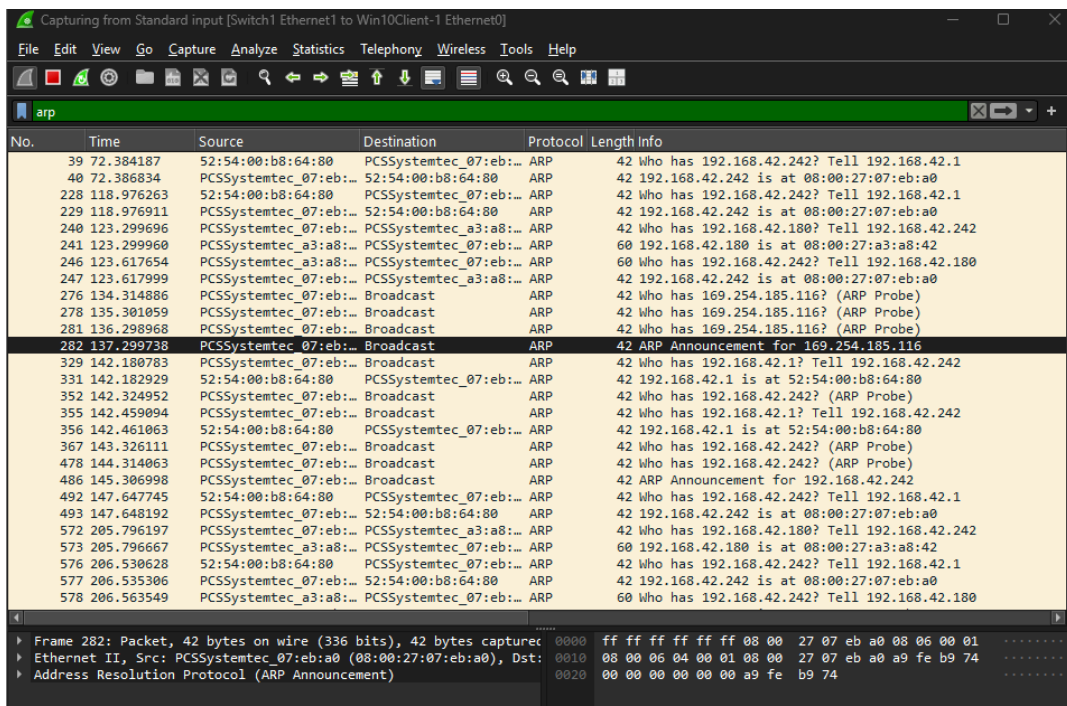
Screenshot 13 ICMP(Ping)



No.	Time	Source	Destination	Protocol	Length	Info
315	139.137896	192.168.42.242	192.168.42.242	ICMP	62	Echo (ping) request id=0x8184, seq=0/0, ttl=64 (no response yet)
823	436.665084	192.168.42.242	172.253.135.113	ICMP	74	Echo (ping) request id=0x0001, seq=17/4352, ttl=128 (replied to 172.253.135.113)
824	436.736822	172.253.135.113	192.168.42.242	ICMP	74	Echo (ping) reply id=0x0001, seq=17/4352, ttl=102 (replied to 192.168.42.242)
825	437.677764	192.168.42.242	172.253.135.113	ICMP	74	Echo (ping) request id=0x0001, seq=18/4608, ttl=128 (replied to 172.253.135.113)
826	437.743613	172.253.135.113	192.168.42.242	ICMP	74	Echo (ping) reply id=0x0001, seq=18/4608, ttl=102 (replied to 192.168.42.242)
828	438.712021	192.168.42.242	172.253.135.113	ICMP	74	Echo (ping) request id=0x0001, seq=19/4864, ttl=128 (replied to 172.253.135.113)
829	438.742315	172.253.135.113	192.168.42.242	ICMP	74	Echo (ping) reply id=0x0001, seq=19/4864, ttl=102 (replied to 192.168.42.242)
830	439.716660	192.168.42.242	172.253.135.113	ICMP	74	Echo (ping) request id=0x0001, seq=20/5120, ttl=128 (replied to 172.253.135.113)
831	439.735559	172.253.135.113	192.168.42.242	ICMP	74	Echo (ping) reply id=0x0001, seq=20/5120, ttl=102 (replied to 192.168.42.242)

Figure 13. ICMP Echo Request and Echo Reply packets captured in Wireshark. The Windows 10 client successfully communicates with an external host, verifying network connectivity.

Screenshot 14 ARP



No.	Time	Source	Destination	Protocol	Length	Info
39	72.384187	52:54:00:b8:64:80	PCSSystemtec_07:eb:a0	ARP	42	Who has 192.168.42.242? Tell 192.168.42.1
40	72.386834	PCSSystemtec_07:eb:a0	52:54:00:b8:64:80	ARP	42	192.168.42.242 is at 08:00:27:07:eb:a0
228	118.976263	52:54:00:b8:64:80	PCSSystemtec_07:eb:a0	ARP	42	Who has 192.168.42.242? Tell 192.168.42.1
229	118.976911	PCSSystemtec_07:eb:a0	52:54:00:b8:64:80	ARP	42	192.168.42.242 is at 08:00:27:07:eb:a0
240	123.299696	PCSSystemtec_07:eb:a0	PCSSystemtec_a3:a8	ARP	42	Who has 192.168.42.180? Tell 192.168.42.242
241	123.299960	PCSSystemtec_a3:a8	PCSSystemtec_07:eb:a0	ARP	60	192.168.42.180 is at 08:00:27:a3:a8:42
246	123.617654	PCSSystemtec_a3:a8	PCSSystemtec_07:eb:a0	ARP	60	Who has 192.168.42.242? Tell 192.168.42.180
247	123.617999	PCSSystemtec_07:eb:a0	PCSSystemtec_a3:a8	ARP	42	192.168.42.242 is at 08:00:27:07:eb:a0
276	134.314886	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	Who has 169.254.185.116? (ARP Probe)
278	135.301059	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	Who has 169.254.185.116? (ARP Probe)
281	136.298968	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	Who has 169.254.185.116? (ARP Probe)
282	137.299738	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	ARP Announcement for 169.254.185.116
329	142.180783	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	Who has 192.168.42.1? Tell 192.168.42.242
331	142.182929	52:54:00:b8:64:80	PCSSystemtec_07:eb:a0	ARP	42	192.168.42.1 is at 52:54:00:b8:64:80
352	142.324952	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	Who has 192.168.42.242? (ARP Probe)
355	142.459094	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	Who has 192.168.42.1? Tell 192.168.42.242
356	142.461063	52:54:00:b8:64:80	PCSSystemtec_07:eb:a0	ARP	42	192.168.42.1 is at 52:54:00:b8:64:80
367	143.326111	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	Who has 192.168.42.242? (ARP Probe)
478	144.314063	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	Who has 192.168.42.242? (ARP Probe)
486	145.306998	PCSSystemtec_07:eb:a0	Broadcast	ARP	42	ARP Announcement for 192.168.42.242
492	147.647745	52:54:00:b8:64:80	PCSSystemtec_07:eb:a0	ARP	42	Who has 192.168.42.242? Tell 192.168.42.1
493	147.648192	PCSSystemtec_07:eb:a0	52:54:00:b8:64:80	ARP	42	192.168.42.242 is at 08:00:27:07:eb:a0
572	205.796197	PCSSystemtec_07:eb:a0	PCSSystemtec_a3:a8	ARP	42	Who has 192.168.42.180? Tell 192.168.42.242
573	205.796667	PCSSystemtec_a3:a8	PCSSystemtec_07:eb:a0	ARP	60	192.168.42.180 is at 08:00:27:a3:a8:42
576	206.530628	52:54:00:b8:64:80	PCSSystemtec_07:eb:a0	ARP	42	Who has 192.168.42.242? Tell 192.168.42.1
577	206.535306	PCSSystemtec_07:eb:a0	52:54:00:b8:64:80	ARP	42	192.168.42.242 is at 08:00:27:07:eb:a0
578	206.563549	PCSSystemtec_a3:a8	PCSSystemtec_07:eb:a0	ARP	60	Who has 192.168.42.242? Tell 192.168.42.180

Frame 282: Packet, 42 bytes on wire (336 bits), 42 bytes captured on interface
Ethernet II, Src: PCSSystemtec_07:eb:a0 (08:00:27:07:eb:a0), Dst: PCSSystemtec_a3:a8 (08:00:27:07:eb:a0)
Address Resolution Protocol (ARP Announcement)

Figure 14. ARP requests and replies captured in Wireshark. The Windows 10 client resolves the MAC addresses of devices on the local network, including the default gateway and the Ubuntu server.

Screenshot 15 TCP Three-way handshake

Capturing from Standard input [Switch1 Ethernet1 to Win10Client-1 Ethernet0]

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp

No.	Time	Source	Destination	Protocol	Length	Info
1077	672.274069	4.153.29.52	192.168.42.242	TCP	131	[TCP Retransmission] 443 → 64483 [FIN, PSH, ACK] Seq=1:
1080	676.281174	104.208.203.88	192.168.42.242	TCP	99	[TCP Retransmission] 443 → 64450 [ACK] Seq=1 Ack=1 Win:
1082	678.282068	23.64.115.217	192.168.42.242	TCP	85	[TCP Retransmission] 443 → 64481 [FIN, PSH, ACK] Seq=1:
1083	679.282526	23.64.115.217	192.168.42.242	TCP	85	[TCP Retransmission] 443 → 64482 [FIN, PSH, ACK] Seq=2:
1087	684.289086	4.153.29.52	192.168.42.242	TCP	131	[TCP Retransmission] 443 → 64483 [FIN, PSH, ACK] Seq=1:
1090	688.294144	104.208.203.88	192.168.42.242	TCP	99	[TCP Retransmission] 443 → 64450 [ACK] Seq=1 Ack=1 Win:
1092	690.296181	23.64.115.217	192.168.42.242	TCP	85	[TCP Retransmission] 443 → 64481 [FIN, PSH, ACK] Seq=1:
1093	691.297971	23.64.115.217	192.168.42.242	TCP	54	443 → 64482 [RST, ACK] Seq=257 Ack=582 Win=65535 Len=0
1100	694.745108	192.168.42.242	34.223.124.45	TCP	66	64321 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256
1101	695.756248	192.168.42.242	34.223.124.45	TCP	66	[TCP Retransmission] 64321 → 80 [SYN] Seq=0 Win=64240
1105	697.259762	4.153.29.52	192.168.42.242	TCP	131	[TCP Retransmission] 443 → 64483 [FIN, PSH, ACK] Seq=1:
1106	697.789017	192.168.42.242	34.223.124.45	TCP	66	[TCP Retransmission] 64321 → 80 [SYN] Seq=0 Win=64240
1113	701.600211	104.208.203.88	192.168.42.242	TCP	54	443 → 64450 [RST, ACK] Seq=152 Ack=1 Win=65535 Len=0
1114	701.790532	192.168.42.242	34.223.124.45	TCP	66	[TCP Retransmission] 64321 → 80 [SYN] Seq=0 Win=64240
1115	701.823569	34.223.124.45	192.168.42.242	TCP	58	80 → 64321 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=
1116	701.824303	192.168.42.242	34.223.124.45	TCP	54	64321 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
1117	701.824994	192.168.42.242	34.223.124.45	HTTP	130	GET / HTTP/1.1
1118	701.825216	34.223.124.45	192.168.42.242	TCP	54	80 → 64321 [ACK] Seq=1 Ack=77 Win=65535 Len=0
1120	702.011139	34.223.124.45	192.168.42.242	TCP	1514	80 → 64321 [PSH, ACK] Seq=1 Ack=77 Win=65535 Len=1460
1121	702.011231	34.223.124.45	192.168.42.242	TCP	1514	80 → 64321 [ACK] Seq=1461 Ack=77 Win=65535 Len=1460 [T
1122	702.013733	192.168.42.242	34.223.124.45	TCP	54	64321 → 80 [ACK] Seq=77 Ack=2921 Win=64240 Len=0
1123	702.016890	34.223.124.45	192.168.42.242	HTTP	1395	HTTP/1.1 200 OK (text/html)
1124	702.020909	192.168.42.242	34.223.124.45	TCP	54	64321 → 80 [FIN, ACK] Seq=77 Ack=4262 Win=62899 Len=0
1125	702.021074	34.223.124.45	192.168.42.242	TCP	54	80 → 64321 [ACK] Seq=4262 Ack=78 Win=65535 Len=0
1126	702.095241	34.223.124.45	192.168.42.242	TCP	54	80 → 64321 [FIN, ACK] Seq=4262 Ack=78 Win=65535 Len=0
1127	702.095663	192.168.42.242	34.223.124.45	TCP	54	64321 → 80 [ACK] Seq=78 Ack=4263 Win=62899 Len=0
1129	704.098754	23.64.115.217	192.168.42.242	TCP	85	[TCP Retransmission] 443 → 64481 [FIN, PSH, ACK] Seq=1:

```
C:\Windows\system32>curl http://neverssl.com
<html>
  <head>
    <title>NeverSSL - Connecting ... </title>
    <style>
      body {
        font-family: Montserrat, helvetica, arial, sans-serif;
        font-size: 16px;
        color: #444444;
        margin: 0;
      }
      h2 {
        font-weight: 700;
        font-size: 1.6em;
        margin-top: 30px;
      }
      p {
        line-height: 1.6em;
      }
      .container {
```

Figure 15. TCP three-way handshake followed by an HTTP GET request and HTTP 200 OK response. The client establishes a reliable TCP connection before requesting a web page from the server.

7. Challenges and Encounters

In order to create the virtual network, it was necessary to face a number of obstacles, which had to be overcome in order for all services to start working properly. During the entire course of the project, different networking parts have been tested individually to identify problems and make sure that all services worked properly.

One of the first difficulties was associated with the virtual machine setup and its ability to communicate inside the GNS3. In the beginning, there were some problems with the networking and adapters' configuration of virtual machines, as a result of which the devices did not manage to communicate properly.

Configuring the Ubuntu DNS server was another challenge encountered during this project. Following installation and configuration of BIND9, the DNS zone files were edited and checked to enable the Windows client to resolve the hostname `server.project.local` using the `nslookup` and `ping` commands.

There was a need to ensure proper timing to capture the appropriate network traffic using Wireshark. This is because DHCP packets only exist when the address is being assigned to the computer; therefore, capture had to start before renewing the client's IP address to capture the entire discovery of the DORA cycle. Various display filters including `dhcp`, `dns`, `icmp`, `arp`, and `tcp` were employed.

Finally, Internet connectivity through the GNS3 NAT Cloud was verified to ensure the virtual network could reach external websites while maintaining communication with the internal DNS and web server. Testing each protocol individually made troubleshooting easier and confirmed that every required network service was functioning as intended.

8. Conclusion

This project was able to show the creation, setup, and testing of a virtual network using GNS3. This virtual network contained the Windows 10 client, an Ubuntu server which acts as the DNS and web services server, an Ethernet switch, and a NAT cloud for Internet services. In totality, this made up the working virtual network.

The whole project saw the Windows client getting its network settings via DHCP, resolving hostnames via the BIND9 DNS server, accessing the web server in Ubuntu via HTTP, and communicating with internal and external hosts using ICMP. Wireshark captures were able to validate the proper interaction of the protocols used such as DHCP, DNS, ARP, ICMP, TCP, UDP, HTTP, NAT, and IP addressing.

This project helped me gain better insights into how various networking services function in a real-life scenario. This project also allowed me to learn hands on how to configure network services and troubleshoot problems related to connectivity along with analysis of packet capture using Wireshark. Overall, the project was successful in achieving its goals.